Applied Software Development Lab
Lab Handout 6: Moving pictures

Learning objectives
- **Python basics: tuples**
- **First steps with PyGame**
- **Further steps toward object-oriented programming**

Step 1: Cloning your own repository

Today we work with Python only. The microcontroller will come back into play later, though (in the truest sense of the word). So that your code does not get lost, first clone your repository.

As always:
- Open two terminals (left for editing, right for running commands).

Left terminal (editing)

A) **Clone the repository and change the directory**
   Clone the repository:
   `git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm`

B) **Change the directory:**
   `cd asp-ulm/python`

Right terminal (running commands)

A) **Change the directory**
   `cd asp-ulm/python`

Step 2: Python basics (lists vs. tuples)

Goal: In the next step, we will use the PyGame module for drawing. PyGame often uses so-called **tuples** as a data structure – for example, for the size of the window or the position of a graphical object.
Before we really get going with PyGame, let us get familiar with these **tuples** and how they differ from **lists**.

Preparation: In the right window, start a Python shell with the command `python`

1. **Assignment with several variables**

   Enter the following in the shell:
   (1) `x, y = 100, 200`
   (2) `x`
   (3) `y`
   You should observe that the first statement assigns the values `100` and `200` to `x` and `y`, respectively:
   ```
   >>> x, y = 100, 200
   >>> x
   100
   >>> y
   200
   ```
2. **Lists: a mutable data structure**

   Now we create a list `pos` and play around with it a little:
   (1) `pos = [x,y]`
   (2) `pos[0]`
   (3) `pos[1]`
   (4) `pos`
   (5) `pos.append(42)`
   (6) `pos`
   As you already know from the online Python course, with a list you can:
   - access individual elements
   - add elements

   In this example, you observe:
   ```
   >>> pos = [x,y]
   >>> pos[0]
   100
   >>> pos[1]
   200
   >>> pos
   [100, 200]
   >>> pos.append(42)
   >>> pos
   [100, 200, 42]
   ```

© Michael Lehn | CC BY-SA 4.0

3. Tuples: an immutable data structure

Now we create a **tuple** `pos` and observe that you can work with it in a similar way as with a **list**. However, a **tuple** only supports operations that do *not modify* `pos`. This is why **tuples** are often used to represent fixed values (e.g., the size of a window or a fixed position).

Enter the following in the shell:

```
(1) pos = (x,y)
(2) pos[0]
(3) pos[1]
(4) pos
(5) pos[0] = 3
(6) pos.append(42)
```

This is what happened:
- With (1), the **tuple** `pos` with two elements was created (for this, you use round instead of square brackets).
- In (2) and (3), the elements were accessed – this works exactly as with a **list**.
- In (5), an attempt was made to modify the first element – this is *not allowed* for a **tuple**.
- For the same reason, `.append()` is not supported either, so a tuple cannot be extended.

Example output:

```
>>> pos = (x,y)
>>> pos[0]
100
>>> pos[1]
200
>>> pos
(100, 200)
>>> pos[0] = 3
Traceback (most recent call last):
  File "<python-input-27>", line 1, in <module>
    pos[0] = 3
    ~~~^^^
TypeError: 'tuple' object does not support item assignment
>>> pos.append(42)
Traceback (most recent call last):
  File "<python-input-28>", line 1, in <module>
    pos.append(42)
    ^^^^^^^^^^
AttributeError: 'tuple' object has no attribute 'append'
```

Conclusion:
- **Tuples** are suitable for *fixed values* that are not supposed to change in the program.
- In *PyGame*, for example, you will often use tuples to pass the window size or a fixed position.

© Michael Lehn | CC BY-SA 4.0

Step 3: First steps with PyGame

Goal: In this step, we will use the **SDL** library (*Simple DirectMedia Layer*).
SDL is a library written in C that allows direct access to the **graphics card**, the **sound**, and other **hardware components**. This makes it ideal for programming games or other graphical applications. However, we will use **SDL not directly**, but through the **Python library PyGame**. **PyGame** is a so-called **wrapper** for SDL: it provides us with a convenient Python interface that lets us use SDL very easily and efficiently.
In this step, we will first learn:

- how to create a window of a **specific size** with PyGame, and
- how to draw a **graphical object** in it (here: a yellow filled circle as the "ball").

The main point is to understand how a typical PyGame program is **structured**:

- Such a program usually consists of an **initialization** and a **main loop**, in each iteration of which a **scene** is drawn.
- This is the **basic structure** that we will extend step by step later – for example, **to move** the ball.

Preparation:

In the left terminal, use the editor of your choice to create a file `breakout.py` with the following content. (The empty lines are intentional – we will fill them in bit by bit in the tasks.)

```
 1 import pygame
 2
 3
 4 # setup
 5 pygame.init()
 6
 7 width, height = 800, 600
 8 screen = pygame.display.set_mode((width, height))
 9
10 ball_c = (255, 255, 0)
11 ball_x, ball_y = 400, 300
12
13 # draw stuff
14
15 pygame.draw.circle(screen, ball_c, (ball_x, ball_y), 5)
16
17
18
19 # done
20 pygame.quit()
```

Tasks

1. Run the program in the right terminal with the command `python breakout.py`. A new window should appear briefly, but it disappears again in a flash.
2. So far, there has been no explanation of what the individual program lines mean. But that is not very interesting at the moment anyway, as long as you cannot see anything in the window.
   So let us first make sure that the window does not disappear immediately:
   1. In **line 2**, insert the following line: `import time`
   2. In **line 17**, insert the following line: `time.sleep(5)`

© Michael Lehn | CC BY-SA 4.0

If you now start the program again with `python breakout.py`, the window stays visible for 5 seconds.
However, the window is still black and has no visible content.
To make the visible result a little more interesting, insert the following line in **line 16**:
`pygame.display.flip()`
Run the program once more. Now, during those 5 seconds, you should see a yellow circle in the middle of the new window.

What the program does:

- With `import pygame`, the module `pygame` is imported.
- With `pygame.init()`, the necessary preparations are made so that you can work with the graphics card, the keyboard, or the sound card at all.
- With `pygame.display.set_mode`, a window is created that you can then draw into. The width and height of the window are passed to the function – as a so-called tuple, i.e., a pair of numbers such as `(800, 600)`.
  The function returns a **variable** that you can later use to draw into this window. More precisely, it is a so-called **object** of type *Surface* – which, in the end, is also nothing but a special variable that **pygame** uses internally to manage the window. For us it is enough to know: this variable will later be passed as the first argument to drawing functions such as `pygame.draw.circle`.

- With `pygame.draw.circle`, you can draw a circle. The function expects four arguments:
  - the variable that represents the window (i.e., *where* to draw),
  - the color of the circle – here an RGB tuple such as `(255, 255, 0)` for yellow,
  - the center of the circle – as a pair of numbers `(x, y)` for the position on the screen,
  - the radius of the circle – i.e., how large it should be.

- **pygame** (or rather the underlying **SDL**) uses a technique called **double buffering**. You can think of it as two drawing surfaces: one of them is visible, the other one is “behind the scenes”. You always draw on the hidden surface, and when everything is finished, `pygame.display.flip()` switches to the new surface. This prevents you from watching the program while it draws – with animations, this would otherwise lead to ugly flickering or stuttering.

- At the end, the program calls `pygame.quit()`. This is not strictly necessary, because everything is cleaned up anyway when the program terminates. But it is good style to release the resources occupied by pygame.

To conclude this step:

Add the file to the repository, create a commit, and push it with the following commands:

```
git add breakout.py
git commit -a -m "first steps with pygame"
git push
```

© Michael Lehn | CC BY-SA 4.0

Step 4: Structure of a PyGame program and moving pictures

Goal:
Our PyGame program should not simply run for a certain number of seconds, but until it is terminated by the user – either by closing the window (by clicking the `X` at the top right) or by pressing a specific key, for example `Q`.

Preparation:
Below you see slightly adapted code for `breakout.py` with **didactic empty lines** that we will fill in step by step in the tasks. The main change is the `while` **loop** in line 16. It runs as long as the variable `running` has the value `True`. In this version, `running` is always `True` for now. So the statements in lines 17 to 28 are executed over and over again in an **infinite loop**. Another change is that in line 28, after the drawing surface has been made visible (`pygame.display.flip()`), a pause of **1/30 second** has been added. As a result, the picture is updated **at about 30 frames per second (FPS)**. Here is the prepared code:

```
 1 import pygame
 2 import time
 3
 4 # setup
 5 pygame.init()
 6
 7 width, height = 800, 600
 8 screen = pygame.display.set_mode((width, height))
 9
10 ball_c = (255, 255, 0)
11 ball_x, ball_y = 300, 300
12
13
14 # draw stuff
15 running = True
16 while running:
17
18
19
20
21
22
23
24     pygame.draw.circle(screen, ball_c, (ball_x, ball_y), 5)
25
26
27     pygame.display.flip()
28     time.sleep(1/30)
29
30 # done
31 pygame.quit()
```

Tasks

1. Adapt `breakout.py` as shown above and test it in the right terminal. The program now runs in an **infinite loop** and, for the moment, can only be stopped by **aborting** it in the terminal with `Control-C`.

© Michael Lehn | CC BY-SA 4.0

2. To make it possible to terminate the program by closing the window, insert the following lines. Make sure that the lines are indented so that they belong to the while block of the loop in line 16:

```
17      for event in pygame.event.get():
18          if event.type == pygame.QUIT:
19              running = False
```

The function `pygame.event.get()` returns a **list of so-called events**. An event represents an occurrence – for example, closing a window, pressing a key, etc. Test the program before you get a first **crash course in "What is an object?"** afterwards.

At this point, it can hardly be kept secret any longer: events are represented as **objects**. So far, we have mostly avoided the term object and talked about variables instead. That is not wrong either – an object is basically a **special kind of variable**.

What makes objects special? An object can consist **of several components**. You can think of it as a **bundle of values** that belong together. Each of these components can in turn be a variable or another object. The common term for these components is **attributes**.

One such **attribute** of an event object is, e.g., `type`. If an event represents the **closing of a window**, this attribute has the value `pygame.QUIT`. In this case, the code above causes `running` to be set to `False` – as a result, the **outer while loop** is exited, and the program terminates.

3. When a key is pressed on the keyboard, this creates an **event object** whose attribute type has the value `pygame.KEYDOWN`. Another attribute, `key`, stores which key was pressed. For example, the value `pygame.K_q` represents the key **Q**.
Change the program as follows:

```
17      for event in pygame.event.get():
18          if event.type == pygame.QUIT:
19              running = False
20          elif event.type == pygame.KEYDOWN:
21              if event.key == pygame.K_q:
22                  running = False
```

Now it should also be possible to terminate the program by pressing the **Q** key.

4. To get some **movement** into the game, we change the **position of the ball** in every frame. First, we define a **velocity** for the ball in the x and y directions:

```
12 ball_vx, ball_vy = -1, 2
```

The unit of velocity is "**pixels per frame**", because with the lines

```
25      ball_x += ball_vx
26      ball_y += ball_vy
```

the **position** of the ball is updated after about **1/30 second** each time. Test the program after this change – the remaining **cosmetic flaw** will be fixed next.

5. As it moves, the ball still leaves traces. The solution is simple: when you draw a new frame, you start by clearing the canvas. Concretely, in our case this means that we color it completely black. The RGB value for black is (0,0,0), and the following line colors the drawing surface black:

```
23      pygame.Surface.fill(screen, (0, 0, 0))
```

The ball should now move without leaving any traces.

© Michael Lehn | CC BY-SA 4.0

6. At the moment, the ball still flies off **into nowhere**. With the next change, it should be reflected at the **edges** of the window. From now on, we no longer fill gaps in the program, but **add new lines**.
   Add the following lines directly **after line 26**:

```
27      if ball_x < 0 or ball_x > width:
28          ball_vx *= -1
29      if ball_y < 0 or ball_y > height:
30          ball_vy *= -1
```

   The reflection is not perfect yet, but good enough for now:
   - If the ball has left the playing field on the **left or right**, the velocity in the **x direction** is reversed.
   - If the ball has left the playing field at the **top or bottom**, the velocity in the **y direction** is reversed.

   For a **perfect reflection**, you would not only have to adjust the velocity, but also set the **position** correctly.
   But that is simple math – we can add it later.

7. The ball flies and bounces – great! Now we still need a **paddle**, i.e., a **rectangle**. This is a nice opportunity to repeat and vary what we have just learned. In PyGame, a **paddle** is simply a **rectangle** that is described by:
   - the coordinates of the **top left** corner and
   - a **width** and a **height**.

   Of course, you can also give a rectangle a **color**. To do this, we first create a few **variables**. Add the following lines **after line 12** (analogous to the variables for the ball):

```
13 paddle_c = (0, 0, 255)
14 paddle_x, paddle_y = 300, 570
15 paddle_w, paddle_h = 100, 30
```

   The paddle should be drawn in every frame. To do this, insert the following lines **after line 27** (i.e., **after the ball has been drawn**):

```
28      pygame.draw.rect(screen, paddle_c,
29                       (paddle_x, paddle_y, paddle_w, paddle_h))
```

8. Of course, the **paddle** should **be movable**. Later, we will read the **serial interface** for this and will then be able to change the position by turning the potentiometer. For today, the **left** and **right** arrow keys will do for steering. We already check in the code whether the **Q** key has been pressed. Extend this check as follows:

```
23          elif event.type == pygame.KEYDOWN:
24              if event.key == pygame.K_q:
25                  running = False
26              elif event.key == pygame.K_LEFT:
27                  paddle_x -= 10
28              elif event.key == pygame.K_RIGHT:
29                  paddle_x += 10
```

   Test the program after this change. You should notice two things that are still annoying:
   - The paddle can be moved out of the window.
   - If you hold down an arrow key, the position does not change continuously – you have to release the key and press it again each time.

© Michael Lehn | CC BY-SA 4.0

9. It would be more convenient if, when you **hold down an arrow key**, the paddle kept moving in the corresponding direction until you release the key again. To do this, first remove the lines inserted in the previous subtask (in my case, these were lines **26 to 29**).

Keyboard events are only generated when a key is **pressed** or **released** – **not** while a key is being **held down**. If there were events for that, far too many events would be generated.

However, you can query **which keys are currently being held down** and then check whether the **arrow keys** are among them.
To do this, insert the following lines **after the for loop**. It is important that these lines belong **to the while loop**, but **not to the for loop** for the events! After all, the lines should also be executed when there is no event.
→ So pay close attention to the **indentation depth** (in my case, 4 characters).

```
26      keys = pygame.key.get_pressed()
27      if keys[pygame.K_LEFT]:
28          paddle_x -= 10
29      if keys[pygame.K_RIGHT]:
30          paddle_x += 10
```

With the function `get_pressed()` in line 26, you get a so-called **dictionary** in `keys`. This data structure is covered in more detail in the Python bridging course. Here it is enough to know that you can use it much like a **list** – except that the **index** does not have to be a number.

For example, here `keys[pygame.K_LEFT]` and `keys[pygame.K_RIGHT]` access entries of `keys`. These values are either `True` or `False` – depending on whether the corresponding key is currently **pressed or not**.

10. Now we also solve the problem that the **paddle** can currently still be moved **too far to the left** or **to the right**.
This is easy to do with the following **fix**, which you should insert **after the position of the paddle may have been changed**:

```
31      if paddle_x < 0:
32          paddle_x = 0
33      if paddle_x + paddle_w > width:
34          paddle_x = width - paddle_w
```

To conclude this step:

Add the file to the repository, create a commit, and push it with the following commands:

```
git commit -a -m "ball and paddle hack"
git push
```

What comes next

At this point, you could simply keep working on the game:
- For example, make sure that the **paddle** actually **makes the ball bounce off**.
- Or that you **lose the ball** when it flies out of the playing field at the bottom.

Unfortunately, though, I will play the **spoilsport** a little at this point – and in the next steps show you instead **how to organize the code better**.
To do this, we introduce a **class** `Paddle` and a class `Ball`.
With this structure, further features can later be implemented **much more easily and clearly**.

© Michael Lehn | CC BY-SA 4.0

Step 5: The class Paddle

Our **paddle** consists of **data** and **operations** that manipulate this data:

- **Data** are the variables: `paddle_x`, `paddle_y`, `paddle_w`, `paddle_h`, `paddle_c`.
- **Operations** are the code that moves the paddle **to the left or right**, as well as the code **to draw** the paddle.

In the current version, all of this is scattered all over the code. You may remember from the last lab handout that **global variables** can be encapsulated in a **class**. And you can also put **functions** into the class that manipulate these variables.

Applied to the paddle, this could look like this:

```
14 class Paddle:
15     c = (0, 0, 255)
16     w, h = 100, 30
17     x, y = (width - w) // 2, 570
18
19     def left():
20         Paddle.x -= 10
21         if Paddle.x < 0:
22             Paddle.x = 0
23
24     def right():
25         Paddle.x += 10
26         if Paddle.x + Paddle.w > width:
27             Paddle.x = width - Paddle.w
28
29     def draw(surface):
30         pygame.draw.rect(surface, Paddle.c,
31                          (Paddle.x, Paddle.y, Paddle.w, Paddle.h))
```

Replace the lines in which `paddle_x`, `paddle_y`, `paddle_w`, `paddle_h`, and `paddle_c` were previously initialized with this code.

Once more, to **clarify** what this code does:

- It initializes the global variables `Paddle.c`, `Paddle.x`, `Paddle.y`, `Paddle.w`, `Paddle.h`. You could also say that it defines the class attributes `c`, `x`, `y`, `w`, `h` of `Paddle`.
- It defines the **functions** `Paddle.left` and `Paddle.right`, which modify these variables so that the paddle moves to the **left** or **right**.
- It defines the **function** `Paddle.draw`, which draws the paddle at its current position on a drawing surface.
- The functions defined in the class could also be called the **class methods** `left`, `right`, and `draw` of `Paddle`.

Task: Make use of it!

Change the rest of the program so that the **paddle** is controlled via the **class methods**:

- if the paddle is to **move to the left**, call `Paddle.left()`.
- if the paddle is to **move to the right**, call `Paddle.right()`.
- And call `Paddle.draw(screen)` to draw the paddle on the drawing surface `screen`.

This does not make the program better (the ball still flies through the paddle), but it makes it more readable. If everything works:

```
git commit -a -m "paddle class"
git push
```

© Michael Lehn | CC BY-SA 4.0

Step 6: Basics of classes with multiple instances

For the **ball**, we could – as for the paddle – also define a **class**. We now take the opportunity to introduce so-called **instantiable classes**. That sounds **fancy** at first, but it is quite simple:

It just means that you can create **several objects** from such a class.
Or, to put it another way:
→ There can be **several balls** – but only one **paddle**.

We first explore the **basic idea** in the **interactive Python shell**. Start it in the **right terminal** with the command `python` and first enter the following to define an **"empty" class** Ball:

```
class Ball:
    pass
```

Then:
(1) `ball = Ball()`
(2) `print(type(ball))`
(3) `ball.x = 42`
(4) `print(ball.x)`

You should observe:
- With (1), a new variable (aka **object**) `ball` was defined. As (2) shows, it has the type `Ball`.
- In (3), an **attribute** `x` was added to the object `ball` and initialized with `42`.
- In (4), this attribute `x` is output.

You can also put functions into a class that
- attach attributes to the object,
- manipulate them, or
- read them.

First, enter the following:

```
class Ball:
    def init(ich, x):
        ich.x = x
    def print(ich):
        print(f"mein x ist {ich.x}")
```

Then
(1) `ball = Ball()`
(2) `Ball.init(ball, 123)`
(3) `Ball.print(ball)`

You should observe:
- Now `ball.x` is initialized with `123`.
- The attribute `x` is then output.
- The advantage: here, the attribute `x` is only used and **manipulated inside the functions**.

© Michael Lehn | CC BY-SA 4.0

Now we create a **second ball** and this time call the functions `init` and `print` differently:
(1) `ball2 = Ball()`
(2) `ball2.init(456)`
(3) `ball2.print()`

You should observe:
- `ball2.init(456)` has the same effect as `Ball.init(ball2, 456)`.
- `ball2.print()` has the same effect as `Ball.print(ball2)`.

In fact, this is exactly how Python **transforms** it internally:
For the call `ball2.print()`, Python automatically turns it into `Ball.print(ball2)` in the background.

Now we try to create a third ball – but forget to initialize it before we output it. Enter:
(1) `ball3 = Ball()`
(2) `ball3.print()`

You will see: this results in an **error**.
Because `ball3.x` has not been set yet – this time we did not call `init()`.

Wouldn't it be a good idea if **initialization** happened **automatically** as soon as you create a new ball?
→ Someone actually had this idea already – and it is built right into **Python**!

Enter the following (or use `Arrow-Up` to bring back the old code for the class `Ball` and modify it):

```
class Ball:
    def __init__(ich, x):
        ich.x = x
    def print(ich):
        print(f"mein x ist {ich.x}")
```

Then enter:
(1) `ball3 = Ball(321)`
(2) `ball3.print()`

Now you see:
The initialization with `__init__` happens automatically as soon as you create a new object `Ball(321)`. So you no longer have to call init manually – Python takes care of that **automatically**.

Usually, you do not use `ich` but `self` as the **first parameter** of the functions (also called **object methods**); it specifies the object whose attributes are **initialized**, **manipulated**, or **read**.

However, this is only a **convention** – in theory, you could choose any other name.
What is **not negotiable**, though, is that the function for automatic initialization (also called **construction**) must be named `__init__`.

© Michael Lehn | CC BY-SA 4.0

Step 7: The real class Ball

The concept of a **class** from which you can construct **several instances** can now be implemented for our game as follows:

```
10 class Ball:
11     def __init__(self, x, y, vx, vy):
12         self.x = x
13         self.y = y
14         self.vx = vx
15         self.vy = vy
16         self.c = (255, 255, 0)
17
18     def move(self):
19         self.x += self.vx
20         self.y += self.vy
21         if self.x < 0 or self.x > width:
22             self.vx *= -1
23         if self.y < 0 or self.y > height:
24             self.vy *= -1
25
26     def draw(self, surface):
27         pygame.draw.circle(surface, self.c, (self.x, self.y), 5)
```

Use it to replace the lines in which the variables `ball_c`, `ball_x`, `ball_y`, `ball_vx`, and `ball_vy` were previously initialized.

Brief explanation of the class:

- With the constructor `__init__`, you can create a new **Ball object** and specify its **initial position** and **initial velocity**. Internally, the corresponding **attributes** are attached to the object – plus an **attribute for the color** with a **default value**.
- With the method `move`, a Ball object can be **moved**. At the edges, it is **reflected** as before.
- With the method `draw`, a Ball object can be drawn on a drawing surface.

Task: Make use of it!

- Every feature you program should also be **used**. So if we have a **class** from which you can create **more than one instance**, then you should create at least **two instances**! To do this, insert the following line **before the while loop**:

  ```
  49 balls = [Ball(400, 300, -1, 2), Ball(200, 100, 4, 2)]
  ```

  The **list** `balls` now contains two **Ball objects** with different **initial positions** and **initial velocities**.
- When the balls are **drawn**, you can **move them right afterwards** as well. To apply this to **all Ball objects**, you can iterate over the elements of `balls`:

  ```
  66     for ball in balls:
  67         ball.draw(screen)
  68         ball.move()
  ```

  Use it to replace the line that previously **drew the ball**. Also remove the **old code** that previously moved the ball (i.e., code that still used `ball_x`, `ball_y`, `ball_vx`, `ball_vy`).

- If everything works:

  ```
  git commit -a -m "ball class"
  git push
  ```

© Michael Lehn | CC BY-SA 4.0

Step 8 (Chill out)

Extend the program so that a new **ball** enters the playing field when the key `A` is pressed.

This time you deliberately get **little help**.

Either you already have an idea or even know how to do it –
or you use newfangled stuff like the **Internet**, which offers things like **Google** or **ChatGPT**.

You just have to ask the right questions, for example:

- How do I check in PyGame whether the A key has been pressed?
- How do I append an object to a list?
- etc.

The **new ball** can always have the same **initial position** or **initial velocity** – or you choose them **randomly**.
And if you think of any other fun additions: **Go wild!**

When you are done – into the repository with it:

```
git commit -a -m "fun with objects"
git push
```

© Michael Lehn | CC BY-SA 4.0